Personal Notes on Regression

Tommaso Facchin

January 2026

Contents

1 Linear Regression

Linear regression is one of the most fundamental and widely used techniques in supervised learning. It models the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a linear equation to the observed data. The main idea is to predict the target as a weighted sum of the input features plus an intercept term.
Mathematically, the model can be expressed as:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon$$

Here, β_0 is the intercept, $\beta_1, \ldots, \beta_n$ are the coefficients representing the influence of each feature on the target, and ϵ is the error term capturing the difference between observed and predicted values. **Note that ϵ_i represents the true (unobservable) random noise in the data-generating process, while in practice we work with the residuals $e_i = y_i - \hat{y}_i$, which are the estimated errors after fitting the model.** The coefficients can be interpreted as the expected change in the target variable for a one-unit change in the corresponding feature, assuming all other features remain constant.
Parameter estimation is usually performed using **Ordinary Least Squares (OLS)**, which finds the coefficients that minimize the sum of squared differences between the predicted and actual target values:

$$\text{Minimize } \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

This ensures the best possible linear fit to the training data in terms of squared error.
Linear regression relies on several key **assumptions** that ensure the model produces valid, interpretable, and reliable results. Understanding these assumptions is crucial because if they are violated, the predictions, inference, or coefficient interpretations can be misleading.
- **Linearity**: This assumption states that the relationship between each feature and the target variable is linear. In other words, the expected change in the target is proportional to a change in the feature. If the true relationship is non-linear, linear regression may underfit the data, leading to biased predictions. Visualizing scatter plots of each feature against the target can help detect non-linearity. - **Independence**: Observations should be independent of each other. This means that the value of one observation does not influence another. Violations occur in time series data or grouped data where correlations exist between samples. Ignoring dependence can result in underestimated standard errors and misleading significance tests. - **Homoscedasticity**: The variance of residuals (errors) should be constant across all levels of the features. If residuals systematically increase or decrease with the predicted values (heteroscedasticity), the model may be less efficient and confidence intervals or hypothesis tests can be inaccurate. Plotting residuals versus predicted values is a common way to check for this. - **Normality of residuals**: The residuals should follow a normal distribution. This assumption is especially important for conducting inference, such as calculating confidence intervals or p-values for coefficients. Even if predictions can still be reasonably accurate, non-normal residuals can invalidate hypothesis testing. Histograms or Q-Q plots of residuals are often used to check normality. - **No multicollinearity**: Features should not be highly correlated with each other. When multicollinearity exists, it becomes difficult to isolate the individual effect of each feature on the target, coefficients can become unstable, and small changes in the data can lead to large changes in estimates. Correlation matrices or variance inflation factors (VIF) are commonly used to detect multicollinearity.
Evaluating a linear regression model is essential to understand how well it predicts the target variable and how much of the underlying variability it captures. Unlike classification problems where metrics like accuracy or F1-score are used, regression requires measures that quantify the **difference between predicted and actual values**, as well as the overall explanatory power of the model.
One of the most common metrics is the **Mean Squared Error (MSE)**, which calculates the average of the squared differences between predicted and actual values:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

Here, y_i represents the actual target value, $\hat{y}_i$ is the predicted value, and m is the number of samples. Squaring the errors penalizes larger deviations more heavily, making MSE sensitive to outliers. A lower MSE indicates better predictive performance.
The **Root Mean Squared Error (RMSE)** is simply the square root of the MSE:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

RMSE is often preferred because it is expressed in the same units as the target variable, making it easier to interpret and compare with the scale of the data.
Another important metric is the **Coefficient of Determination (R^2)**:

$$R^2 = 1 - \frac{\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{m}(y_i - \bar{y})^2}$$

R^2 measures the proportion of variance in the target variable that is explained by the model. A value of $R^2 = 1$ indicates a perfect fit, whereas $R^2 = 0$ suggests that the model does no better than predicting the mean of the target. Negative values can occur when the model performs worse than simply predicting the mean.
Together, these metrics give a **comprehensive view** of model performance: MSE and RMSE focus on prediction errors, while R^2 provides insight into how well the model captures the underlying structure of the data. By analyzing these metrics, we can assess not only how accurate our predictions are but also how much of the variability in the target is being explained.
In practice, these evaluation measures guide decisions on model selection, hyperparameter tuning, and whether further feature engineering or preprocessing is needed. Even though linear regression is conceptually simple, mastering these metrics is crucial for building a solid foundation before moving on to more complex supervised learning models.

1.1 Multiple Linear Regression

So far, the linear regression model has been introduced with one generic target variable and several predictors, but the structure is the same whether we have a single feature or many. Multiple linear regression explicitly considers the case where the target depends on several input variables simultaneously.
Mathematically, the multiple linear regression model with $p-1$ predictors can be written as:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_{p-1} x_{i,\,p-1} + \epsilon_i, \quad i = 1, \ldots, m$$

Here, β_0 is the intercept, β_j for $j = 1, \ldots, p-1$ are the slope coefficients associated with each predictor x_{ij}, and ϵ_i are the error terms. As before, ϵ_i represent the unobservable random noise, while the residuals $e_i = y_i - \hat{y}_i$ are their empirical estimates obtained after fitting the model.
In matrix form, the model can be written compactly as:

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

where $\mathbf{y} \in \mathbb{R}^m$ is the vector of responses, $X \in \mathbb{R}^{m \times p}$ is the design matrix (each row corresponds to an observation and each column to a predictor, with the first column typically equal to 1 for the intercept), $\boldsymbol{\beta} \in \mathbb{R}^p$ is the vector of coefficients, and $\boldsymbol{\epsilon} \in \mathbb{R}^m$ is the vector of errors.
Ordinary Least Squares (OLS) estimation in the multiple linear regression setting is obtained by minimizing the sum of squared residuals:

$$\min_{\boldsymbol{\beta}} \sum_{i=1}^{m}(y_i - \hat{y}_i)^2 \quad \text{with} \quad \hat{y}_i = \beta_0 + \sum_{j=1}^{p-1} \beta_j x_{ij}.$$

In matrix notation, the OLS estimator $\hat{\boldsymbol{\beta}}$ has the closed-form solution:

$$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y},$$

provided that $X^\top X$ is invertible. This formula shows that multiple linear regression is still a linear model in the parameters: even if we add more predictors, or transformations of the original predictors, the estimation procedure remains the same.
The interpretation of the coefficients in multiple linear regression is **conditional** on the other variables being held fixed. The coefficient β_j measures the expected change in the target variable y for a one-unit increase in the predictor x_j, assuming all other predictors remain constant. This is a key difference from simple linear regression, where there is only one predictor and no need to condition on other variables.
Multiple linear regression is also the natural framework to introduce issues such as multicollinearity, interaction terms, and polynomial features. For example, a polynomial regression in one variable x,

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_k x_i^k + \epsilon_i,$$

can be seen as a multiple linear regression where the predictors are the transformed features $x_i, x_i^2, \ldots, x_i^k$. This perspective emphasizes that, although the relationship between x and y may be non-linear, the model remains linear in the parameters β_j and can still be estimated using the same OLS machinery.

1.2 Implementation in R

In practice, linear regression models are often fitted using the R function `lm()`, which implements Ordinary Least Squares (OLS) estimation for a wide range of model specifications. A multiple linear regression can be defined as:

```
model <- lm(y ~ x1 + x2 + x3, data = df)
```

Here, `y` is the target variable, `x1`, `x2`, and `x3` are the predictors, and `df` is a data frame containing the data. The formula `y  x1 + x2 + x3` specifies that `y` is modeled as a linear combination of the predictors plus an intercept term, and `lm()` internally estimates the coefficients by minimizing the sum of squared residuals, exactly as described in the OLS formulation.
Before fitting the model, it is common practice to inspect the data and, when appropriate, split it into training and test sets or use cross-validation to obtain an unbiased estimate of the model's generalization performance. In many real-world applications, predictors are also standardized (e.g., using `scale()`) to make coefficients comparable in magnitude and to prepare the data for regularized regression methods.
Once the model is fitted, the `summary()` function provides a detailed overview of the results:

```
summary(model)
```

This output includes the estimated coefficients, their standard errors, t-statistics, and p-values, which are used to assess the statistical significance of each predictor under the usual linear regression assumptions. For each coefficient $\hat{\beta}_j$, the standard error $\mathrm{SE}(\hat{\beta}_j)$ measures the estimated variability of $\hat{\beta}_j$ across hypothetical repeated samples. The **t-statistic** is defined as

$$t_j = \frac{\hat{\beta}_j}{\mathrm{SE}(\hat{\beta}_j)},$$

and, under the null hypothesis $H_0 : \beta_j = 0$ and assuming the linear model assumptions hold, t_j approximately follows a t-distribution with $m-p$ degrees of freedom (where m is the number of observations and p the number of parameters, including the intercept). The corresponding **p-value** is the probability, under H_0, of observing a t-value as extreme or more extreme than the one computed from the data. A small p-value (typically below a chosen significance level, such as 0.05) provides evidence against H_0, suggesting that the predictor x_j is significantly associated with the response after controlling for the other variables in the model.
The `summary()` output also reports the Residual Standard Error, the **R-squared** and **Adjusted R-squared**, and an overall F-statistic that tests whether the model explains a significant amount of variance compared to a null model with no predictors.
The **Adjusted R-squared** is particularly important when comparing models with different numbers of predictors. Unlike the regular R-squared, which never decreases when new variables are added, the Adjusted R-squared penalizes unnecessary complexity and increases only if a new predictor improves the model more than would be expected by chance. In addition to R-squared, model selection in linear regression often relies on information criteria such as the **Akaike Information Criterion (AIC)** and the **Bayesian Information Criterion (BIC)**, which balance goodness of fit and model complexity: for both AIC and BIC, **lower values indicate a better trade-off** between fit and parsimony.

1.3 Matrix Form

Linear regression is most naturally expressed in matrix notation. This allows us to write the model for all observations and all predictors in a single compact equation.
Let:

- $\mathbf{y} \in \mathbb{R}^{m\times 1}$ be the vector of target values (one entry per observation).
- $X \in \mathbb{R}^{m\times p}$ be the design matrix, where:
  - the first column is a column of ones (for the intercept),
  - the remaining $p-1$ columns contain the predictor values.
- $\boldsymbol{\beta} \in \mathbb{R}^{p\times 1}$ be the vector of parameters (intercept and coefficients).
- $\boldsymbol{\epsilon} \in \mathbb{R}^{m\times 1}$ be the vector of error terms.

The multiple linear regression model can then be written compactly as:

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

For a given parameter vector $\boldsymbol{\beta}$, the predictions for all m observations are:

$$\hat{\mathbf{y}} = X\hat{\boldsymbol{\beta}}$$

The Ordinary Least Squares (OLS) estimator $\hat{\boldsymbol{\beta}}$ is obtained by minimizing the sum of squared residuals $\|\mathbf{y} - X\boldsymbol{\beta}\|^2$. Under standard conditions (e.g., $X^\top X$ invertible), the closed-form solution is:

$$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}$$

This shows that, regardless of how many predictors we include, the estimation procedure remains the same: only the design matrix X changes when we add, remove, or transform features.

1.4 Example (Two Predictors)

Consider a small example with two predictors x_1 and x_2 and three observations. We model:

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$$

with data:

i	x_{i1}	x_{i2}	y_i
1	1	0	2
2	2	1	4
3	3	2	7

In matrix form, we define:
- Target vector:

$$\mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 7 \end{bmatrix}$$

- Design matrix (first column of ones for the intercept, then x_1 and x_2):

$$X = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 1 & 3 & 2 \end{bmatrix}$$

- Parameter vector:

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

The model is:

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

Using OLS in matrix form, the estimated coefficients are:

$$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}$$

For this example:

$$X^\top X = \begin{bmatrix} 3 & 6 & 3 \\ 6 & 14 & 8 \\ 3 & 8 & 5 \end{bmatrix}, \qquad X^\top \mathbf{y} = \begin{bmatrix} 13 \\ 31 \\ 18 \end{bmatrix}$$

and $(X^\top X)^{-1}$ exists (details of the inversion can be omitted in practice, since it is handled numerically by libraries). By multiplying $(X^\top X)^{-1}$ and $X^\top \mathbf{y}$ we obtain:

$$\hat{\boldsymbol{\beta}} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

so the fitted model is:

$$\hat{y} = 1 + 1 \cdot x_1 + 1 \cdot x_2.$$

This example shows how the compact matrix formula $(X^\top X)^{-1} X^\top \mathbf{y}$ directly produces the intercept and coefficients for a multiple linear regression model with more than one predictor.

1.5 Variance Inflation Factor (VIF)

When using multiple linear regression, one of the key assumptions is the absence of strong multicollinearity among predictors. Multicollinearity occurs when one predictor is highly correlated with a linear combination of the others, making it difficult to disentangle their individual effects on the target and leading to unstable coefficient estimates. The Variance Inflation Factor (VIF) is a commonly used diagnostic to quantify how much the variance of an estimated coefficient is inflated due to multicollinearity.
For each predictor x_j, the VIF is defined in terms of the coefficient of determination R_j^2 obtained by regressing x_j on all the remaining predictors in the model. Specifically, the VIF for x_j is given by

$$VIF_j = \frac{1}{1 - R_j^2}.$$

If x_j is almost perfectly explained by the other predictors, then R_j^2 will be close to 1 and VIF_j will become very large, indicating severe multicollinearity. Conversely, if x_j is nearly uncorrelated with the others, R_j^2 will be close to 0 and VIF_j will be close to 1, suggesting no relevant inflation of the variance.
Interpretation of VIF values is typically based on simple rules of thumb. A value of 1 indicates no detectable multicollinearity, since the variance of the coefficient is not inflated relative to the ideal case with independent predictors. Values between 1 and about 5 are often regarded as indicating moderate correlation that may not require immediate action, depending on the context and the goals of the analysis. Values larger than 5 or 10 are frequently taken as evidence of problematic multicollinearity, signalling that the corresponding predictor is largely redundant and that its coefficient estimate will be highly unstable and difficult to interpret.
In applied work, VIFs are used as part of a broader diagnostic procedure rather than in isolation. Large VIF values may lead to several possible interventions, such as removing or combining highly correlated predictors, introducing regularization (e.g. ridge regression), or rethinking the model specification based on domain knowledge. It is also important to remember that multicollinearity primarily affects the precision and interpretability of individual coefficients, while overall prediction accuracy (for a given data set) can remain relatively unaffected, especially if the multicollinear predictors are kept together in the model.

1.6 Categorical Variables

In many real-world regression problems, not all predictors are numerical. We often have **categorical variables**, such as `city`, `education_level`, or `product_type`. Linear regression, however, operates on numerical inputs, so categorical variables must be encoded into numeric form before they can be included in the model.
A common strategy is **one-hot encoding** (or dummy encoding). Suppose we have a categorical variable `color` with three possible values: `red`, `green`, `blue`. We create three binary indicators:

- $x_{\text{red}} = 1$ if `color = red`, otherwise 0.
- $x_{\text{green}} = 1$ if `color = green`, otherwise 0.
- $x_{\text{blue}} = 1$ if `color = blue`, otherwise 0.

If we include all three dummies together with an intercept, we introduce perfect multicollinearity, because:

$$x_{\text{red}} + x_{\text{green}} + x_{\text{blue}} = 1 \quad \text{for every observation.}$$

To avoid this, one category is taken as a **reference level** and its dummy is omitted (e.g., drop `blue`). The model becomes:

$$\hat{y}_i = \beta_0 + \beta_{\text{red}} x_{i,\text{red}} + \beta_{\text{green}} x_{i,\text{green}} + (\text{other numeric predictors}).$$

In this parameterization:
- β_0 is the expected value of y for the reference category (`blue`) when all other predictors are zero. - β_{red} is the expected difference in y between `red` and `blue`, holding other predictors fixed. - β_{green} is the expected difference in y between `green` and `blue`, holding other predictors fixed.
From the matrix point of view, each dummy variable is just another column of the design matrix X. The only difference is that these columns contain 0/1 values instead of continuous numbers. As long as we avoid including all dummy columns for a categorical variable (i.e., we drop one category), the usual OLS machinery and the closed-form solution still apply without modification.

1.7 Example: Model Formula with Dummy Variables

Suppose we have one numerical predictor `size` and one categorical predictor `color` with three levels: `red`, `green`, `blue`. After dummy encoding (and dropping `blue` as reference), our predictors are:
- x_{size} (numeric) - x_{red} (dummy: 1 if `color = red`, 0 otherwise) - x_{green} (dummy: 1 if `color = green`, 0 otherwise)
The regression model can be written as:

$$\hat{y}_i = \beta_0 + \beta_{\text{size}}\, x_{i,\text{size}} + \beta_{\text{red}}\, x_{i,\text{red}} + \beta_{\text{green}}\, x_{i,\text{green}}.$$

If we look at each color separately:

- For `blue` (reference): $x_{i,\text{red}} = 0$, $x_{i,\text{green}} = 0$.

$$\hat{y}_i \mid \text{blue} = \beta_0 + \beta_{\text{size}}\, x_{i,\text{size}}.$$

- For `red`: $x_{i,\text{red}} = 1$, $x_{i,\text{green}} = 0$.

$$\hat{y}_i \mid \text{red} = (\beta_0 + \beta_{\text{red}}) + \beta_{\text{size}}\, x_{i,\text{size}}.$$

- For `green`: $x_{i,\text{red}} = 0$, $x_{i,\text{green}} = 1$.

$$\hat{y}_i \mid \text{green} = (\beta_0 + \beta_{\text{green}}) + \beta_{\text{size}}\, x_{i,\text{size}}.$$

This makes the interpretation clear:

- β_0 is the intercept for the reference category `blue`.
- β_{red} and β_{green} shift the intercept up or down for `red` and `green` relative to `blue`, while the slope with respect to `size` remains β_{size} for all colors.

1.8 Interaction Terms

So far, the regression models have assumed that each predictor has an effect on the target that does not depend on the values of the other predictors. In many real-world problems, however, the effect of one variable can **change** depending on another variable. Interaction terms allow the model to capture these context-dependent effects.
In a multiple linear regression without interactions, a model like

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$$

assumes that: - the effect of x_1 on y is always β_1, regardless of the value of x_2 - the effect of x_2 on y is always β_2, regardless of the value of x_1.
To allow the effect of x_1 to depend on x_2, we add an **interaction term** $x_{i1}x_{i2}$:

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \gamma\, x_{i1}x_{i2}.$$

Now the partial effect of x_1 on $\hat{y}$ is

$$\frac{\partial \hat{y}_i}{\partial x_{i1}} = \beta_1 + \gamma x_{i2},$$

so it changes with x_{i2}. Similarly, the effect of x_2 depends on x_1.
From a design-matrix perspective, the interaction term is simply a **new column** in X, whose entries are the products $x_{i1}x_{i2}$.

1.9 Example: Numeric × Categorical

Consider the earlier example with one numerical predictor `size` and one categorical predictor `color` with levels `red`, `green`, `blue` (with `blue` as reference). After dummy encoding, we have:
- x_{size} (numeric) - x_{red} (dummy) - x_{green} (dummy)
A model **without** interactions is:

$$\hat{y}_i = \beta_0 + \beta_{\text{size}}\, x_{i,\text{size}} + \beta_{\text{red}}\, x_{i,\text{red}} + \beta_{\text{green}}\, x_{i,\text{green}}.$$

In this model, the slope with respect to `size` is β_{size} for all colors; the dummies shift only the intercept.
To allow the slope of `size` to depend on `color`, we include interaction terms between `size` and the dummies:

$$\hat{y}_i = \beta_0 + \beta_{\text{size}}\, x_{i,\text{size}} + \beta_{\text{red}}\, x_{i,\text{red}} + \beta_{\text{green}}\, x_{i,\text{green}} + \gamma_{\text{red}}\, (x_{i,\text{size}} \cdot x_{i,\text{red}}) + \gamma_{\text{green}}\, (x_{i,\text{size}} \cdot x_{i,\text{green}}).$$

Now the model implies different lines for each color:
- `blue` (reference): $x_{i,\text{red}} = x_{i,\text{green}} = 0$

$$\hat{y}_i \mid \text{blue} = \beta_0 + \beta_{\text{size}}\, x_{i,\text{size}}.$$

- `red`: $x_{i,\text{red}} = 1$, $x_{i,\text{green}} = 0$

$$\hat{y}_i \mid \text{red} = (\beta_0 + \beta_{\text{red}}) + (\beta_{\text{size}} + \gamma_{\text{red}})\, x_{i,\text{size}}.$$

- `green`: $x_{i,\text{red}} = 0$, $x_{i,\text{green}} = 1$

$$\hat{y}_i \mid \text{green} = (\beta_0 + \beta_{\text{green}}) + (\beta_{\text{size}} + \gamma_{\text{green}})\, x_{i,\text{size}}.$$

In this parameterization: - the intercept changes across colors (through β_{red} and β_{green}), - the slope with respect to `size` also changes across colors (through γ_{red} and γ_{green}).
Interaction terms are useful when:
- domain knowledge suggests that the effect of one variable should depend on another (e.g., effect of a treatment depending on age, effect of price depending on segment), - plots or exploratory analysis show that the relationship between a numeric predictor and y differs across groups.
In practice, interactions are implemented by adding product columns to the design matrix X. The model remains linear in the parameters and can still be estimated with the usual OLS machinery.

1.10 ANOVA

In the context of multiple linear regression, Analysis of Variance (ANOVA) decomposes the total variability of the response variable into a component explained jointly by all predictors in the model and a residual (unexplained) component. The starting point is the decomposition of the total sum of squares:

$$\text{TSS} = \sum_{i=1}^{m}(y_i - \bar{y})^2 = \underbrace{\sum_{i=1}^{m}(\hat{y}_i - \bar{y})^2}_{\text{Model sum of squares (MSS)}} + \underbrace{\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}_{\text{Residual sum of squares (RSS)}}$$

Here, TSS (Total Sum of Squares) measures the overall variability of y around its mean, MSS measures the variability explained by the multiple regression model (i.e., by all predictors taken together), and RSS measures the variability that remains in the residuals. Note that the coefficient of determination can be written as $R^2 = \text{MSS}/\text{TSS}$.
In a multiple linear regression with p parameters (including the intercept), the degrees of freedom associated with each sum of squares are:
- Model degrees of freedom: $\text{df}_{\text{model}} = p - 1$ (number of slope coefficients/predictors) - Residual degrees of freedom: $\text{df}_{\text{res}} = m - p$ - Total degrees of freedom: $\text{df}_{\text{tot}} = m - 1$
From these, the mean squares are defined as:

$$\text{MS}_{\text{model}} = \frac{\text{MSS}}{\text{df}_{\text{model}}}, \quad \text{MS}_{\text{res}} = \frac{\text{RSS}}{\text{df}_{\text{res}}}$$

The overall F-test for the multiple regression model compares the full model (with all predictors) against a null model with only the intercept. The null and alternative hypotheses are:
- H_0: all slope coefficients are zero (the predictors, taken together, do not explain more variability than the mean), i.e. $\beta_1 = \beta_2 = \cdots = \beta_{p-1} = 0$ - H_1: at least one slope coefficient is non-zero (the predictors, taken together, explain a non-negligible part of the variability)
The F-statistic is defined as:

$$F = \frac{\text{MS}_{\text{model}}}{\text{MS}_{\text{res}}}$$

Under H_0, this statistic approximately follows an F-distribution with $(\text{df}_{\text{model}}, \text{df}_{\text{res}})$ degrees of freedom. A large value of F (with a small p-value) provides evidence against H_0, indicating that the multiple regression model explains a significant portion of the variability in the response compared to using only the mean.
In R, the ANOVA table for a single multiple regression model is obtained via:

```
anova(model)
```

which reports, for each source of variation, the sum of squares (SS), degrees of freedom (Df), mean squares (MS), F-statistics, and p-values.
ANOVA can also be used to compare **nested multiple regression models**, for example, a reduced model containing only a subset of predictors and a more complex full model with additional predictors:

```
model_reduced <- lm(y ~ x1 + x2, data = df)
model_full    <- lm(y ~ x1 + x2 + x3, data = df)

anova(model_reduced, model_full)
```

In this case, the hypotheses are:

- H_0: the reduced model is sufficient (the additional predictor(s) in `model_full` do not improve the fit).
- H_1: the full model provides a better fit (at least one of the additional predictors contributes significantly).

The F-statistic is constructed from the reduction in RSS and the loss of degrees of freedom when moving from the reduced to the full model. A significant p-value indicates that the additional predictors in the full multiple regression model lead to a statistically significant improvement in explained variance.

1.11 AIC and BIC

While ANOVA focuses on hypothesis testing for nested models, information criteria such as the Akaike Information Criterion (AIC) and the Bayesian Information Criterion (BIC) provide a way to compare multiple models (not necessarily nested) by balancing goodness of fit and model complexity.
For a fitted model with maximized log-likelihood ℓ and k free parameters, the AIC is defined as:

$$\text{AIC} = 2k - 2\ell$$

and the BIC is defined as:

$$\text{BIC} = k\log(m) - 2\ell$$

where m is the number of observations. Both criteria penalize models with more parameters:
- The term $2k$ in AIC and $k\log(m)$ in BIC act as penalties for model complexity. - The term -2ℓ decreases as the model fits the data better (higher log-likelihood).
In linear regression with Gaussian errors, the log-likelihood ℓ is, up to an additive constant, a function of the residual sum of squares (RSS):

$$\ell \propto -\frac{m}{2}\log\left(\frac{\text{RSS}}{m}\right)$$

so models with smaller RSS (better fit) tend to have larger ℓ and therefore smaller AIC and BIC, all else being equal.
The general rule for model comparison is:
- Prefer the model with **lower AIC**. - Prefer the model with **lower BIC**.
Because BIC uses $\log(m)$ instead of 2 as penalty weight, it penalizes complexity more strongly than AIC, especially for large sample sizes. As a result:
- AIC tends to favor models with better predictive performance, possibly with slightly more parameters. - BIC tends to favor more parsimonious models and is often used in a more "conservative" model selection perspective.
In R, AIC and BIC can be computed directly:

```
AIC(model_reduced, model_full)
BIC(model_reduced, model_full)
```

These functions return the AIC/BIC values for each model, allowing direct comparison. When combined with ANOVA and adjusted R-squared, AIC and BIC provide a comprehensive toolkit to decide which linear regression specification is most appropriate in terms of both explanatory power and complexity.

1.12 Residuals vs Fitted plot

A key diagnostic tool for linear regression is the residuals vs fitted plot, which visualizes the relationship between the residuals $e_i = y_i - \hat{y}_i$ and the fitted values $\hat{y}_i$. Under the standard linear regression assumptions, the residuals should be centered around zero and display no systematic pattern as a function of the fitted values.
If the model is appropriate and the assumptions of linearity and homoscedasticity hold, the residuals vs fitted plot should show a random cloud of points with roughly constant vertical spread along the horizontal axis. In contrast, visible structures (for example, a curved pattern or a "funnel" shape with increasing spread) suggest model misspecification: curvature indicates that the linearity assumption may be violated, while a funnel-shaped pattern indicates heteroscedasticity (non-constant variance of the errors).
In R, the residuals vs fitted plot can be obtained directly from a fitted `lm` object:

```
plot(model, which = 1)
```

Here, `which = 1` selects the first standard diagnostic plot, where the x-axis shows the fitted values $\hat{y}_i$ and the y-axis shows the standardized residuals. This visualization is particularly useful to decide whether transformations of the response or predictors, or a more flexible model (e.g. adding polynomial terms or interactions), might be necessary to better capture the underlying relationship.

1.13 Q-Q plot of residuals

Another important assumption of the classical linear regression model is that the errors (and therefore the residuals, as their estimates) are approximately normally distributed. This assumption is crucial for the validity of t-tests, F-tests, and confidence intervals for the regression coefficients. A Q-Q (quantile–quantile) plot provides a visual check of the normality of the residuals.
The idea of the Q-Q plot is to compare the empirical quantiles of the residuals with the theoretical quantiles of a standard normal distribution. If the residuals follow a normal distribution, the points in the Q-Q plot should lie approximately on a straight line. Systematic deviations from the line indicate departures from normality: heavy tails (points bending away at the ends), skewness (points systematically above or below the line on one side), or other irregular patterns.
In R, the Q-Q plot for the residuals of a linear model can be produced with:

```
plot(model, which = 2)
```

This generates a normal Q-Q plot of the standardized residuals. Mild deviations from the line are often acceptable in practice, especially for large samples where the central limit theorem mitigates moderate non-normality. However, strong deviations may suggest the presence of outliers, skewness, or other distributional issues that can affect inference, and might motivate transformations of the response or the use of more robust modeling approaches.

1.14 Bias–Variance Tradeoff

When building regression models, there is a fundamental tradeoff between bias and variance that determines how well the model will generalize to unseen data. Intuitively, increasing model complexity (for example, by adding more predictors or using higher-degree polynomials) tends to reduce bias but increase variance, while simplifying the model has the opposite effect.
In the standard regression setting, the expected prediction error at a point x can be decomposed as:

$$\mathbb{E}\big[(Y - \hat{f}(x))^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(x)] - f(x)\big)^2}_{\text{Bias}^2} + \underbrace{\mathbb{V}[\hat{f}(x)]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible error}}$$

Here, $f(x)$ is the true regression function, $\hat{f}(x)$ is the model's prediction, and σ^2 is the variance of the noise in the data. The **bias** term measures how far, on average, the model's predictions are from the true function (systematic error), while the **variance** term measures how sensitive the model is to fluctuations in the training data (how much the predictions change if we fit the model on different samples).
Simple models, such as a linear regression with very few predictors or low-degree polynomials, typically have **high bias and low variance**: they may underfit the data and fail to capture important structure, but their predictions are relatively stable across different training sets. In contrast, very flexible models, such as high-degree polynomial regressions or models with many interaction terms, tend to have **low bias and high variance**: they can closely follow the training data, including noise, but their predictions may change drastically if the training data change slightly, leading to overfitting.
The bias–variance tradeoff implies that there is an intermediate level of model complexity that minimizes the overall expected prediction error. In practice, techniques such as **cross-validation** are used to select model complexity (for example, the degree of a polynomial or the amount of regularization) that achieves a good balance between bias and variance. Regularization methods like Ridge, Lasso, and Elastic Net explicitly control model flexibility by shrinking coefficients towards zero, thereby reducing variance at the cost of introducing some additional bias.

1.15 Polynomial Regression

Polynomial regression is an extension of linear regression that allows the model to capture non-linear relationships between the input features and the target variable by introducing polynomial terms of the original features. Although the relationship between the original feature x and the target y can be curved, the model remains **linear in the parameters** because it is a linear combination of transformed features.
In the simplest case with a single predictor x, a polynomial regression of degree k can be written as:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_k x_i^k + \epsilon_i, \quad i = 1, \ldots, m$$

If we define new features $z_{i1} = x_i,\ z_{i2} = x_i^2, \ldots, z_{ik} = x_i^k$, this model is just a multiple linear regression in the transformed feature vector $(z_{i1}, \ldots, z_{ik})$.
Let Z be the design matrix whose columns are $\mathbf{1}, x, x^2, \ldots, x^k$. Then the model is:

$$\mathbf{y} = Z\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

and the coefficients can still be estimated by Ordinary Least Squares using:

$$\hat{\boldsymbol{\beta}} = (Z^\top Z)^{-1} Z^\top \mathbf{y},$$

provided that $Z^\top Z$ is invertible. This shows that polynomial regression does not change the estimation machinery; it only changes the feature space in which linear regression is applied.

1.16 Implementing polynomial regression in R

There are two common ways to specify polynomial terms in R:
1. **Using explicit powers with `I()`** (raw polynomial):

```
# quadratic
fit_quad <- lm(y ~ x + I(x^2), data = df)

# cubic
fit_cubic <- lm(y ~ x + I(x^2) + I(x^3), data = df)
```

The `I()` function tells R to interpret `x^2` as *x squared* and not as part of the formula language.
2. **Using `poly()`**:

```
# degree 3, orthogonal polynomials (default)
fit_poly3 <- lm(y ~ poly(x, 3), data = df)

# degree 3, raw powers
fit_poly3_raw <- lm(y ~ poly(x, 3, raw = TRUE), data = df)
```

- `poly(x, 3, raw = TRUE)` generates the columns x, x^2, x^3 (like manual `I(x^2)`, etc.).
- `poly(x, 3)` (default `raw = FALSE`) generates **orthogonal polynomials**, which are linear combinations of x, x^2, x^3 but decorrelated; this improves numerical stability. The fitted values are the same up to numerical precision, only the coefficients differ in interpretation.

1.17 Box–Cox Transformation

The Box–Cox transformation is a parametric family of power transformations for strictly positive responses $y > 0$, defined as:

$$y^{(\lambda)} = \begin{cases} \dfrac{y^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \log(y), & \lambda = 0 \end{cases}$$

The parameter λ controls the shape of the transformation: $\lambda = 1$ corresponds to no transformation, $\lambda = 0$ to a log transform, values like $\lambda \approx 0.5$ approximate a square-root transform, and $\lambda = -1$ is close to a reciprocal transform. The main goals are to reduce skewness and heteroscedasticity and to make the relationship between predictors and response closer to linear.
In the context of linear regression, the Box–Cox transformation is typically applied to the **response** y rather than the predictors. One considers a model of the form

$$y_i^{(\lambda)} = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \epsilon_i,$$

where $y_i^{(\lambda)}$ is the transformed response. The parameter λ is chosen (for example) by maximizing the profile log-likelihood under the assumption of normal, constant-variance errors for $y^{(\lambda)}$. After selecting λ, the linear model is fitted on the transformed scale using ordinary least squares, and standard diagnostic tools (residual plots, Q–Q plots) are used to assess whether normality and homoscedasticity have improved.

2 Generalized Linear Model (GLM)

Generalized Linear Models extend ordinary linear regression to handle non-Gaussian responses (binary, counts, positive data, proportions) while preserving a linear predictor and a likelihood-based estimation framework.
A standard linear model assumes a continuous response with normal errors and a linear relation between predictors and the response mean:

$$Y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \epsilon_i, \quad \epsilon_i \sim \text{Normal}(0, \sigma^2).$$

This is often unrealistic for binary outcomes, counts, or skewed positive data. GLMs generalize this by:
- allowing Y_i to follow a distribution from the **exponential family** (e.g. normal, binomial, Poisson, gamma) - keeping a **linear predictor** $\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$ - linking the **mean** $\mu_i = \mathbb{E}[Y_i \mid X_i]$ to η_i through a **link function** g:

$$g(\mu_i) = \eta_i.$$

The classical linear regression model is a special case: normal errors + identity link $g(\mu) = \mu$.
Every GLM is defined by three building blocks:
1. **Random component (distribution of Y)** The conditional distribution of Y_i given predictors belongs to the exponential family: - normal for continuous symmetric data - binomial/Bernoulli for binary or proportion data - Poisson for counts - gamma for positive, continuous, right-skewed responses. The model specifies $Y_i \mid X_i \sim \text{Dist}(\mu_i, \phi)$ where μ_i is the mean and ϕ is a dispersion (or scale) parameter.
2. **Systematic component (linear predictor)** Predictors enter through a linear combination:

$$\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}.$$

Here x_{ij} can be: - raw numeric features - dummy variables for categoricals - interactions, polynomial terms, etc. This is the same structure as in multiple linear regression; only the meaning of η_i changes via the link.
3. **Link function** The link g connects the mean μ_i to the linear predictor:

$$g(\mu_i) = \eta_i \quad \Longleftrightarrow \quad \mu_i = g^{-1}(\eta_i).$$

It is chosen so that: - the transformed mean $g(\mu_i)$ can range over all real numbers (matching $\eta_i \in \mathbb{R}$) - the inverse link $g^{-1}(\cdot)$ maps back into the natural domain of the mean (e.g. $(0, 1)$ for probabilities, $(0, \infty)$ for intensities).
Typical "canonical" links:
- **Normal**: identity link $g(\mu) = \mu$ (ordinary linear regression). - **Binomial**: logit link $g(\mu) = \log\big(\mu/(1-\mu)\big)$, giving logistic regression for binary outcomes. - **Poisson**: log link $g(\mu) = \log(\mu)$, giving a log-linear model for counts. - **Gamma**: inverse or log link, for positive skewed responses.
GLMs allow you to:
- model responses with **non-Gaussian distributions** while keeping a linear predictor - respect the **natural range** of the response (probabilities in $(0, 1)$, counts $\in \{0, 1, 2, \dots\}$, positive data > 0) - maintain a unified framework for: - parameter estimation via **maximum likelihood** - model comparison via **deviance** and **information criteria** (AIC, BIC) - inference using asymptotic normality of estimators (standard errors, Wald tests, likelihood ratio tests).
Generalized linear models in R are fitted with the same formula syntax as `lm()`, using `glm()` and specifying the appropriate family (distribution + link). A generic GLM in R looks like:

```
model_glm <- glm(y ~ x1 + x2 + x1:x2,
                 family = binomial(link = "logit"),
                 data = df)
```

- `y ~ x1 + x2 + x1:x2` encodes the linear predictor $\eta_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i1} x_{i2}$. - `family = binomial(link = "logit")` chooses the binomial distribution and logit link; changing `family` (e.g. `poisson(link = "log")`, `Gamma(link = "log")`) changes the random component and link, while the formula part (the linear predictor) remains the same.
In GLMs, inference on coefficients is still based on standard errors and Wald-type statistics, but, instead of a t-value, software typically reports a z**-value**:

$$z_j = \frac{\hat{\beta}_j}{\text{SE}(\hat{\beta}_j)},$$

which is interpreted using the standard normal distribution (asymptotic approximation) to obtain p-values, rather than a t-distribution as in ordinary least squares.

2.1 Deviance in GLMs

In generalized linear models, the **deviance** is a generalization of the residual sum of squares used in linear regression. It measures how far a fitted model is from the "saturated" model (a model that fits each observation perfectly) in terms of log-likelihood.
For a GLM, the deviance of a fitted model is defined as:

$$D = 2\left[\ell(\text{saturated model}) - \ell(\text{fitted model})\right],$$

where $\ell(\cdot)$ denotes the maximized log-likelihood of the model. Smaller deviance means the fitted model is closer (in likelihood terms) to the saturated model, i.e., it explains the data better.
For normal linear regression with identity link and constant variance, the deviance is (up to a constant factor) just the **residual sum of squares**:

$$D \propto \sum_{i=1}^{m} (y_i - \hat{y}_i)^2,$$

so deviance reduces to the familiar notion of "sum of squared residuals".
Deviance is especially useful for:
- **Goodness of fit**: the residual deviance of a single model (compared to its degrees of freedom) gives a rough idea of how well the model fits. - **Model comparison**: the difference in deviance between two nested models,

$$\Delta D = D_{\text{reduced}} - D_{\text{full}},$$

can be used as a test statistic (likelihood ratio test) to assess whether the additional parameters in the full model significantly improve the fit.
In practice, software reports:
- **Residual deviance**: deviance of the current fitted model. - **Null deviance**: deviance of a model with only the intercept.
The reduction from null deviance to residual deviance is the part of variability explained by the predictors in the GLM, analogous to how the reduction in total sum of squares is interpreted in linear regression.

2.2 Normal GLM (Linear Regression as a GLM)

In the GLM framework, ordinary linear regression corresponds to choosing a normal response and the identity link. Specifically,
- Random component: $Y_i \mid X_i \sim \text{Normal}(\mu_i, \sigma^2)$ - Systematic component:

$$\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$$

- Link function:

$$g(\mu_i) = \mu_i = \eta_i$$

so that $\mu_i = \mathbb{E}[Y_i \mid X_i] = \eta_i$.
This shows that the classical multiple linear regression model is a special case of a GLM with normal errors and identity link. All the extensions discussed earlier (dummy variables, interaction terms, polynomial features) can be seen as instances of the same GLM structure with a different design matrix X, while the distribution and link remain the same.

2.3 Binomial GLM

Binomial GLMs are used for modeling binary outcomes (0/1) or counts of "successes" out of a fixed number of trials, using an appropriate distribution and link function. Typical examples include modeling whether a customer churns (yes/no), whether a patient responds to a treatment (success/failure), or the proportion of defective items in a batch out of n_i inspected units.
In the simplest binary case, each observation Y_i takes values in $\{0, 1\}$ and is modeled as a Bernoulli variable with success probability μ_i:

$$Y_i \sim \text{Bernoulli}(\mu_i), \quad \mathbb{P}(Y_i = 1 \mid X_i) = \mu_i.$$

More generally, for grouped/binomial data, you can have Y_i successes out of n_i trials:

$$Y_i \sim \text{Binomial}(n_i, \mu_i), \quad i = 1, \ldots, m.$$

Here μ_i is the success probability for observation i, and the expected value is $\mathbb{E}[Y_i \mid X_i] = n_i\mu_i$ or $\mathbb{E}[Y_i/n_i \mid X_i] = \mu_i$.

2.4 Link function: logistic regression

A binomial GLM specifies a link between the mean μ_i and a linear predictor $\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$.
The **canonical** link for the binomial family is the **logit**:

$$g(\mu_i) = \log\left(\frac{\mu_i}{1-\mu_i}\right) = \eta_i.$$

Equivalently,

$$\mu_i = g^{-1}(\eta_i) = \frac{\exp(\eta_i)}{1+\exp(\eta_i)} = \frac{1}{1+\exp(-\eta_i)}.$$

This is the standard **logistic regression** model. The key interpretive points:
- η_i is the **log-odds** of success:

$$\eta_i = \log\left(\frac{\mu_i}{1-\mu_i}\right).$$

- A one-unit increase in x_{ij} changes η_i by β_j, so it multiplies the **odds** by $\exp(\beta_j)$:

$$\text{odds}_i = \frac{\mu_i}{1-\mu_i}, \quad \text{odds}_i^{(\text{new})} = \exp(\beta_j) \cdot \text{odds}_i^{(\text{old})}.$$

In binomial GLMs, the three components are:
- Random component: $Y_i \mid X_i \sim \text{Binomial}(n_i, \mu_i)$ (or Bernoulli with $n_i = 1$). - Systematic component:

$$\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}.$$

- Link function (logit):

$$g(\mu_i) = \log\left(\frac{\mu_i}{1-\mu_i}\right) = \eta_i.$$

Parameters β are estimated by **maximum likelihood**. There is no closed-form solution like in OLS; numerical algorithms (iteratively reweighted least squares, IRLS) are used internally, but conceptualmente rimane: trovi i β che rendono i dati osservati più plausibili secondo il modello binomiale con link logit.

```
# binary outcome: y = 1 if customer churns, 0 otherwise
# x1, x2 are predictors (e.g., tenure, monthly_charges)
fit_logit <- glm(
  y ~ x1 + x2,
  family = binomial(link = "logit"),
  data   = df
)

summary(fit_logit)
```

Here the formula specifies the linear predictor $\eta_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$, while family = binomial(link = "logit") chooses the binomial distribution and logit link; the fitted probabilities are $\hat{\mu}_i = \mathbb{P}(Y_i = 1 \mid X_i) = 1/(1+e^{-\hat{\eta}_i})$.

2.5 Poisson GLM

Poisson GLMs model **count data** (number of events) using a Poisson distribution and typically a log link. They are appropriate when the response is a non-negative integer and represents event counts per unit time, area, or other exposure.
In a basic Poisson regression, each observation Y_i is a count:

$$Y_i \sim \text{Poisson}(\mu_i), \quad Y_i \in \{0, 1, 2, \dots\}$$

with mean and variance both equal to μ_i (equidispersion). Typical examples include the number of calls per hour, number of incidents per day, or number of claims per policy.
A Poisson GLM links the mean μ_i to a linear predictor $\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$ via the **log link**:

$$g(\mu_i) = \log(\mu_i) = \eta_i.$$

Equivalently,

$$\mu_i = g^{-1}(\eta_i) = \exp(\eta_i).$$

This guarantees $\mu_i > 0$ for any real η_i and yields a **log-linear model** for the expected count. A one-unit increase in x_{ij} changes η_i by β_j, so it multiplies the expected count by $\exp(\beta_j)$:
- additively on the log scale: $\eta_i^{(\text{new})} = \eta_i^{(\text{old})} + \beta_j$ - multiplicatively on the count scale: $\mu_i^{(\text{new})} = \exp(\beta_j) \cdot \mu_i^{(\text{old})}$.
- **Random component**: $Y_i \mid X_i \sim \text{Poisson}(\mu_i)$
- **Systematic component**:

$$\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$$

- **Link function (log)**:

$$g(\mu_i) = \log(\mu_i) = \eta_i \quad \Longleftrightarrow \quad \mu_i = \exp(\eta_i)$$

Parameters β are estimated by maximum likelihood using the GLM machinery, again via iteratively reweighted least squares.
Often counts are observed over different **exposure** levels (time, population, area). In that case, Poisson regression is used to model **rates** while keeping a Poisson likelihood by including an **offset** term. If t_i is exposure (e.g. time at risk), one models:

$$\log(\mu_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \log(t_i),$$

which is equivalent to modeling the log-rate $\log(\mu_i/t_i)$ as linear in the predictors, with $\log(t_i)$ having a fixed coefficient of 1.

2.6 Gamma GLM

Gamma GLMs model **positive, continuous, right-skewed data** (e.g. waiting times, costs) using a Gamma distribution, typically with a log link. They are appropriate when the response is strictly positive and the variance increases roughly with the square of the mean.
In a basic Gamma regression, each observation Y_i is a positive continuous variable:

$$Y_i \sim \text{Gamma}(\text{mean } \mu_i, \text{shape } k), \quad Y_i > 0,$$

with $\mathbb{E}[Y_i \mid X_i] = \mu_i$ and $\text{Var}(Y_i \mid X_i) \propto \mu_i^2$. Typical examples include insurance claim amounts, hospital length of stay, or positive costs and durations.
A Gamma GLM links the mean μ_i to a linear predictor $\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$ via a **link function**, most commonly the **log link**:

$$g(\mu_i) = \log(\mu_i) = \eta_i.$$

Equivalently,

$$\mu_i = g^{-1}(\eta_i) = \exp(\eta_i).$$

This guarantees $\mu_i > 0$ for any real η_i and yields a **log-linear model** for the expected value. A one-unit increase in x_{ij} changes η_i by β_j, so it multiplies the expected outcome by $\exp(\beta_j)$:
- additively on the log scale: $\eta_i^{(\text{new})} = \eta_i^{(\text{old})} + \beta_j$ - multiplicatively on the original scale: $\mu_i^{(\text{new})} = \exp(\beta_j) \cdot \mu_i^{(\text{old})}$.
(An alternative, less common choice is the **inverse link** $g(\mu_i) = 1/\mu_i$, but in practice the log link is standard.)
- **Random component**: $Y_i \mid X_i \sim \text{Gamma}(\mu_i, k)$, with $\mu_i > 0$.
- **Systematic component**:

$$\eta_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}.$$

- **Link function (usually log)**:

$$g(\mu_i) = \log(\mu_i) = \eta_i \quad \Longleftrightarrow \quad \mu_i = \exp(\eta_i).$$

Parameters β (and often the shape k) are estimated by maximum likelihood using the general GLM machinery (e.g. iteratively reweighted least squares).
The **exponential distribution** is a special case of the Gamma family with shape parameter $k = 1$. An exponential regression model for waiting times can therefore be seen as a Gamma GLM with shape fixed to 1, using the same log-link structure and GLM estimation, but with this specific restriction on the shape parameter.